



PDF Download
3624062.3624176.pdf
29 January 2026
Total Citations: 1
Total Downloads: 360

DL Latest updates: <https://dl.acm.org/doi/10.1145/3624062.3624176>

RESEARCH-ARTICLE

Accelerating In Situ Analysis using Non-volatile Memory

DEEPAK B HEGDE, Indian Institute of Technology Kanpur, Kanpur, UP, India

PREETI MALAKAR, Indian Institute of Technology Kanpur, Kanpur, UP, India

Open Access Support provided by:

Indian Institute of Technology Kanpur

Published: 12 November 2023

[Citation in BibTeX format](#)

SC-W 2023: Workshops of The
International Conference on High
Performance Computing, Network,
Storage, and Analysis
November 12 - 17, 2023
CO, Denver, USA

Accelerating In Situ Analysis using Non-volatile Memory

Deepak B Hegde

deepakbh21@iitk.ac.in

Indian Institute of Technology Kanpur

India

Preeti Malakar

pmalakar@iitk.ac.in

Indian Institute of Technology Kanpur

India

ABSTRACT

The relatively slower data transfer speeds resulting in I/O bottlenecks in scientific simulations is one of the critical challenges in exascale computing. Simulations generate large data and analysis applications consume this data to provide time-critical insights. The limited size and high power consumption of Dynamic Random Access Memory (DRAM) capacity leaves slow storage devices as the primary option for large-scale data transfers. Non-volatile memory (NVM) devices such as Intel Optane bridges the gap between storage and volatile memory by providing DRAM-comparable performance and persistence. We present PQueue, a data transfer library for in situ analysis of simulation output using persistent memory. PQueue leverages NVM and provides an API that resembles high-level parallel I/O libraries such as PnetCDF to enable seamless transition for application developers. We achieved a maximum of 7× improvement in write times and a maximum of 10× improvement in read times as compared to PnetCDF.

ACM Reference Format:

Deepak B Hegde and Preeti Malakar. 2023. Accelerating In Situ Analysis using Non-volatile Memory. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023)*, November 12–17, 2023, Denver, CO, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3624062.3624176>

1 INTRODUCTION

Recent advances in computing speeds have enabled supercomputers process vast amount of data in a few minutes. The fastest supercomputer can compute 10^{18} floating point operations in a second [5]. Thus the read/write bandwidths of the memory and disk have become the limiting factors of performance of an application rather than the compute rates [25] [6] [8]. Unfortunately, dynamic random access memory (DRAM) encounters challenges in terms of scalability and cost [18], thereby posing difficulties in increasing its capacity. Solid state drives (SSDs) and other storage devices exhibit insufficient speed to substitute for dynamic random-access memory (DRAM) effectively. Intel Optane memory [19, 26] serves as a solution that effectively addresses the disparity between high-speed but limited volatile memory and slow yet abundant storage devices [23]. Intel's Optane [26] technology exhibits attributes reminiscent of solid-state drive (SSD) storage and dynamic random-access

memory (DRAM). Specifically, Optane ensures data persistence without needing periodic refreshes while enabling byte addressability, thereby facilitating direct access to data structures.

In situ analysis of scientific simulations [9] is one of the applications that may benefit the most from using non-volatile memory (NVM) for staging data [10]. The conventional workflow of scientific simulations such as weather forecasting simulations [20] involve the execution of simulation and analysis tasks. The simulation tasks execute numerical simulations resulting in raw output. The analysis jobs are responsible for processing the output data, offering meteorologists and end users valuable insights regarding the simulations. The simulations and analysis are sometimes time-sensitive. A parallel file system is mostly used to transfer data between simulation and analysis applications. However, using storage media such as SSDs or hard disk drives poses significant limitations in these workflows due to their relatively slower speeds as compared to DRAM. The output data volume is often too large to be accommodated within the limited space of DRAM memory. Thus non-volatile memory (NVM) is a feasible solution for efficient in situ analysis. In this work, we propose PQueue (persistent queue) for simulations and analysis processes to stage data using persistent memory using similar methods as conventional high-level parallel I/O libraries. PQueue enhances the efficiency of in situ simulation data analysis by utilizing a persistent memory-based queue. The design of PQueue is simple and can be directly used by applications that write NetCDF files. PQueue will utilize NVM (instead of the file system) and thus automatically speed up such applications.

The objective of this work is to enable simulation processes (writers) seamlessly write data efficiently into the persistent memory while the analysis processes (readers) may read data independently. Concurrent access by readers and writers to the parallel file system often leads to significant input/output (I/O) bottleneck. We are faced with a similar challenging situation even in case of persistent memory [1, 3] due to limitations of PMDK which disallows access of the same memory pool by multiple processes. The contention arises due to the simultaneous access of the memory pool by both the reader and writer processes. We designed a novel approach in PQueue that involves partitioning the memory pool into individual entries for each time step and organizing them within a data structure. This arrangement enables the simulation application to generate new entries in the queue without being delayed by the analysis process.

PQueue may be integrated with any simulation application that uses NetCDF-like high-level parallel I/O library effortlessly by dynamic linking. The calls to PnetCDF/NetCDF will be automatically replaced with calls to PQueue in order for the application to use memory instead of disk by dynamic linking with PQueue. PnetCDF is a sophisticated parallel input/output (I/O) library explicitly designed to read and write scientific data collections efficiently. We

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SC-W 2023, November 12–17, 2023, Denver, CO, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0785-8/23/11...\$15.00

<https://doi.org/10.1145/3624062.3624176>

provide an Application Programming Interface (API) that resembles PnetCDF [17] for read/write functions. This enables application developers to easily integrate PQueue to use NVM for the application because of its similarity with the PnetCDF methods. We achieved a maximum improvement of $10\times$ in read times and a maximum improvement of $7\times$ in write times as compared to the PnetCDF library on a custom-designed micro-benchmark. We tested with various data sizes and several combinations of reader to writer ratios.

2 RELATED WORK AND BACKGROUND

In this section, we describe some of the prior work in this area and provide a brief overview of PnetCDF functions.

2.1 Related Work

Relatively slower data transfer speeds in disks and increasing gap between computation and I/O performance speeds have led researchers to explore various alternatives to decrease the I/O gap. In situ analysis is one of the prominent research directions towards improving the end-to-end analysis time of scientific simulations. Traditionally, analyzing scientific simulations involves the simulation process writing data to the disk and the analysis process fetching data from the disk, resulting in two I/O accesses. In situ analysis often bypasses the I/O bottleneck by analyzing or visualizing data as soon as it is available without writing to disk [9, 10]. There have been approaches for various scientific simulations where analysis applications directly read data from the simulation application’s memory (DRAM) or by accessing a staging area where the simulation application writes its output data [15] [24] [10] [7]. Its most beneficial to stage the data on memory which is closest to the simulation process. However, DRAM capacity is limited. In this work, we present a library that uses non-volatile memory on each node as the intermediate storage for scientific data analysis.

Jin et al. [15] explore multi-tiered data staging using DRAM and SSD for in situ/in-transit processing. However, they focus on explicitly managing data placements across the hierarchy. Subedi et al. [24] explore using burst buffers for data staging in case of in situ workflows. They focus on autonomous data movement across DRAM and SSDs by predicting data access sequence. In contrast, we have developed a library, PQueue, that resembles existing parallel scientific libraries such as PnetCDF and utilize non-volatile memory storage to improve the performance of analysis applications seamlessly.

Works leveraging persistent memory storage capabilities focus on developing a generic data structure [12] or addressing a specific part of the scientific data [16]. Though there are works related to utilizing non-volatile memory for developing specific scientific applications [21] [14], there is no generic standard library that the application developers can utilize to leverage non-volatile memory capabilities for data transfers in scientific application. In order to leverage non-volatile memory capabilities for in situ analysis and develop a generic framework for in situ processing, we propose PQueue, an end-to-end, high-speed data transfer library designed for in situ analysis of scientific simulations.

2.2 Persistent Memory

Persistent memory (PMEM) is an emerging memory technology that combines the critical attributes of dynamic random-access memory (DRAM) and solid-state drives (SSDs). This technology possesses the dual characteristics of non-volatility and direct accessibility by the central processing unit (CPU) through load and store instructions, resembling the functionality of DRAM. Intel Optane has two distinct modes. Firstly, it can be deployed in memory mode, physically inserted into the Dual In-line Memory Module (DIMM) slots. Memory mode directly allows the CPU to access the Optane module as the primary memory. Secondly, Optane supports App Direct mode, which involves using a PMEM-Aware file system to facilitate access to the Optane module.

Intel provides a Software Development Kit (SDK) known as the Persistent Memory Development Kit (PMDK) [3]. This SDK has various APIs that facilitate allocating and modifying memory pools from persistent memory. The software employs file system programming interfaces to generate files and establish memory mappings. The persistence of changes made by store instructions is guaranteed when flushed to storage. Once memory mappings are established for persistent memory, the application can directly access the non-volatile memory at the byte level using load and store instructions. By leveraging the capability to directly access non-volatile memory (NVM) at the byte level, storing data in a manner that closely resembles memory access is possible.

2.3 PnetCDF Methods

NetCDF (Network Common Data Form) is a collection of software libraries and machine-independent data formats that facilitate creating, accessing, and exchanging array-based scientific data [22]. It is also a standard for the exchange of scientific data. The Unidata Program Center supports and maintains C, C++, Java, and Fortran NetCDF programming interfaces. PnetCDF [17] is a high-performance parallel I/O library. The objective is to attain excellent I/O performance. Due to the implementation of MPI, PnetCDF employs a new set of APIs starting with the prefix `ncmpi_*` similar to the NetCDF APIs `nc_*`.

PnetCDF is widely used in various scientific simulation workflows for the purpose of storing scientific data [17]. PnetCDF functions can be divided into following categories based on their utility.

- (1) Dataset functions: Perform operations such as creating, opening, closing, or aborting a dataset.
- (2) Define Mode functions: These functions are used to specify the dimensions and variables of a dataset.
- (3) Inquiry functions: Provide information about the dataset’s metadata, such as the dimensions and variables present, their names, lengths, data types, and the number of dimensions, variables, and attributes.
- (4) Attribute functions: Enable the management of dataset attributes, including adding, modifying, and retrieving attribute values.
- (5) Data Mode functions: Enable reading from and writing to dataset variables using different access methods. These methods include accessing a single element, the entire array, a subarray, a subsampled array (using strides), and a mapped strided subarray.

We have developed APIs that are similar to PnetCDF. The resemblance allows applications relying on PnetCDF to seamlessly utilize PQueue without modifying their existing code.

3 PERSISTENT QUEUE (PQUEUE)

Scientific simulations produce a large amount of output data which is required to be analyzed and visualized for comprehending the simulation output. Traditional simulations use parallel I/O libraries such as NetCDF [22] and HDF5 [11] to write the output to the disk. The analysis and visualization routines read the data back from the disk. This post hoc simulation-analysis pipeline incurs I/O cost twice. Contemporary I/O bandwidth is still lagging behind memory bandwidth and compute rates. Thus we propose a library, PQueue, to enable read/write to non-volatile memory (NVM) to avoid disk I/O cost for analysis of the simulation output. This will help in faster analysis and visualization of data from memory. In this section, we discuss the design requirements of PQueue, followed by the implemented design of PQueue.

3.1 PQueue Design Requirements

We have developed a library, PQueue, to utilize the advantages of non-volatile memory for in situ parallel analysis of scientific simulations [9]. The objective is to reduce the turnaround time of the simulation-analysis pipeline. PQueue is designed so that simulation processes may write their data to non-volatile memory and analysis processes may read data from memory as soon as the data is produced. This results in a possibility of concurrent access from both simulation processes (writers) and analysis processes (readers). Thus we designed the library to ensure the correctness of the data read by analysis along with minimizing any locking effects due to shared memory access. We focus on the following aspects:

- (1) Utilize the persistent memory characteristics and byte addressability of non-volatile memory.
- (2) Mitigate conflicts arising from concurrent access to reader and writer applications.
- (3) Minimize source code modifications to existing simulation codes to utilize PQueue.

We describe the PQueue design in the next section.

3.2 PQueue Design

Figure 1 illustrates the preliminary design of PQueue. The simulation process writes to the end of the queue that stores the simulation output data in non-volatile memory (NVM). The middle group of rectangles depict the output time steps. In case of in situ analysis, the analysis job (consumer/reader) may start reading data as soon as the simulation job (producer/writer) completes writing a time step. The simulation job (producer) completes a particular step and subsequently stores the resulting output data in an entry, which is then placed in a *queue* as depicted in Figure 1. The analysis job (consumer) can continuously read from this queue as the producer keeps writing the entries.

The queue entries are based on simulation time steps. The *queue* is responsible for storing the time step entry generated by the simulation process. The light orange box on the right shows the header data (metadata) and variable data (VAR_1 , VAR_2 , ...) stored as part of each queue entry. The analysis process reads from the

beginning of the queue for further processing. We used semaphores to access the shared queue. Each time step entry in the queue stores the corresponding variable information, such as variable offsets, variable sizes, variable data in non-volatile memory (NVM). Variable metadata per queue such as the total number of variables in each queue entry, the number of dimensions is stored as a header in each queue entry.

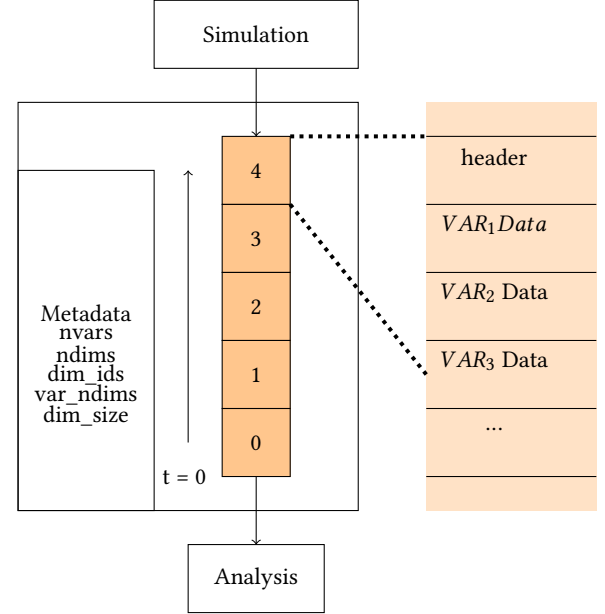


Figure 1: PQueue

In this work, we focused on developing PQueue API similar to NetCDF. Thus we assume that the simulation output will be written in a self-describing format. The scientific metadata is appended to the queue, describing various variables and dimensions of the data. For example, the metadata of a weather simulation would comprise variables such as temperature, pressure and wind velocities among several others, and their respective dimensions. This process is similar to the self-describing file format of NetCDF. The metadata is written only once at the beginning of the simulation. The metadata, also known as a header, includes the starting indices and extents of the variables written to each entry.

Non-volatile memory stores the time step entry in the queue as persistent memory pools. Persistent memory pools are memory-mapped files residing in the persistent memory. Persistent memory exposes byte-addressable memory as a special block device that applications can access from user space [1]. The queue and its entries are stored in persistent memory as memory pools, allowing processes to access them directly by utilizing the appropriate offsets for the variables. Only a producer is allowed to access an entry before it is committed to the queue, as the consumer requires data only after it is written. This effectively prevents simultaneous access to the queue by both readers and writers.

Simulation applications usually run on a large number of processes in parallel. Analysis applications typically run on relatively fewer processes, therefore multiple producers per consumer may

exist. Numerous simulation processes may finish generating simulation data at similar intervals and write the data to the queue. Since analysis processes start processing data as soon as its available, multiple producers and consumers may require access to the queue simultaneously. However, current implementation of PMDK [3] does not support simultaneous access to memory pools by multiple processes [4].

Our initial design has a limitation due to a complete entry in a single pool. This necessitates serialization of all write operations performed by the simulation application processes. It implies that more than one simulation process will not be able to simultaneously write the simulation data to queue entry in a single queue. Additionally, it is essential to note that both the writer and reader applications may require simultaneous access to metadata or pointers associated with various entries. PMDK's limitation also serializes these accesses. In order to address the limitation of PMDK on simultaneous access to memory pools and resolve the contention that arises due to simultaneous accesses by reader and writer processes, we propose a modification to the current design by introducing split PQueue. We describe this in the next section.

3.3 Split PQueue

Scientific simulations partition the entire data domain into smaller subdomains and distribute among the participating simulation processes. These processes solve and compute their local domains and communicate with each other to get relevant information about the other subdomains. For example, weather applications discretize the earth into a 3-dimensional grid distributed over several processes. We redesigned PQueue to enable simultaneous access to persistent memory by multiple processes for scalable in situ analysis.

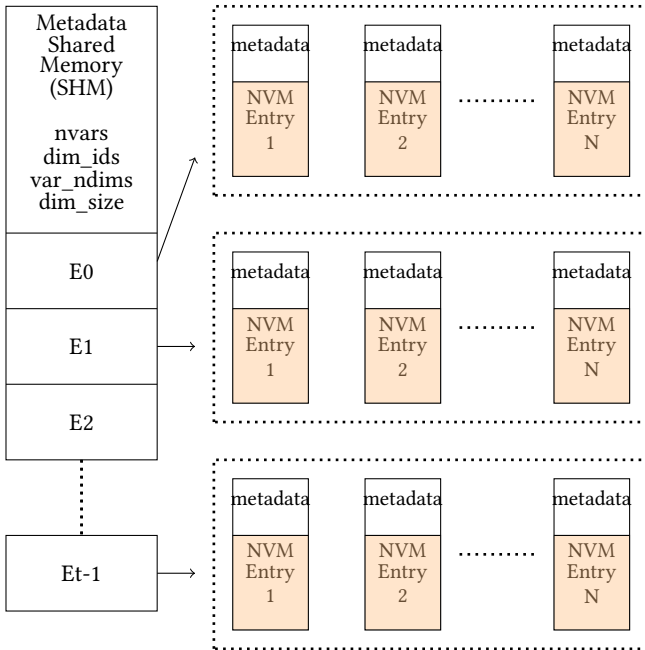


Figure 2: Split-PQueue

The redesigned version of PQueue is illustrated in Figure 2. In the left rectangle of the figure, $E_0, E_1, \dots, E_{t-1}$ represents the t queue entries, where each simulation time step is stored. Each queue entry points to N subdomains, where N represents the total number of processes in the simulation application. The data written by each process to time step entry is called a subdomain entry. The dotted outline on the right connected to each of the entries $E_0, E_1, \dots, E_{t-1}$ shows the subdomain split. Each memory pool stores individual subdomain entries. Hence multiple simulation processes can access the respective memory pools at the same time. Multiple simulation processes may write their respective time step data to the same entry (depicted as $E_0, E_1, \dots$) in the pool, storing a complete entry in a single pool.

Each sub-domain entry consists of a private metadata component that contains relevant details regarding the data contained within that entry. A distinct shared memory segment SHM in DRAM, depicted in the left top rectangle of Figure 2, stores the global metadata accessible to both the simulation and analysis applications concurrently. This SHM segment maps to the address space of each process within the simulation workflow. The split PQueue structure exists on each node where the simulation and analysis processes run. Only those processes that are running on the same node as the split PQueue have access to write to the queue.

We set up a fixed number of entries in order to mitigate the cost associated with establishing an entry pool and allocating storage for each write cycle. This number is configurable. The entries are recorded sequentially within these pools using a round-robin approach. When the number of simulation processes is large, the application may assign specific processes the role of writers. Consequently, the count of subdomains will correspond to the number of writers responsible for aggregating data from the subdomains of non-writer processes. This feature may allow each writer to generate its data in a manner that is entirely independent of the other writer's processes. This is part of our future design.

4 PQQUEUE EXECUTION

We describe the methods implemented in PQueue, the implementation details and the runtime details of using PQueue in this section.

4.1 PQueue API

We designed the PQueue Application Programming Interface (API) such that it closely resembles the read/write methods currently used in scientific simulations such as in PnetCDF/NetCDF [17]. This enables application developers to directly use the corresponding PQueue methods to write to memory in place of the original PnetCDF functions. We also provide library interposition to invoke PQueue methods from any application that originally invokes NetCDF functions. This requires only linking with PQueue and does not entail source code modifications.

We present the functions that we implemented in PQueue as well as the mappings to the corresponding PnetCDF functions in Tables 1 and 2. An overview was also presented in Section 2.3. We show the different categories of the functions, such as dataset functions, define mode functions, and inquire functions in Table 1 and attribute and data mode functions in Table 2. We provide a one-to-one mapping for all relevant NetCDF functions in PQueue

so that the application developers do not need to modify the source code. Before launching the application, developers can preload the PQueue library using the LD_PRELOAD mechanism. Consequently, the applications will use the PQueue library methods by default, superseding the usage of PnetCDF functions.

Table 1: PnetCDF and PQueue Dataset, Define mode and Inquiry APIs

Dataset APIs	
PnetCDF APIs	PQueue APIs
ncmpi_create()	pqueue_create()
ncmpi_open()	pqueue_create(open mode)
ncmpi_enddef()	pqueue_enddef()
ncmpi_sync()	pqueue_flush_entry()
ncmpi_close()	pqueue_close()
Define mode APIs	
ncmpi_def_dim()	pqueue_def_dim()
ncmpi_def_var()	pqueue_def_var()
Inquiry APIs	
ncmpi_inq()	pqueue_inq()
ncmpi_inq_ndims()	pqueue_inq_ndims()
ncmpi_inq_natts()	pqueue_inq_natts()
ncmpi_inq_dimid()	pqueue_inq_dimid()
ncmpi_inq_dim()	pqueue_inq_dim()
ncmpi_inq_dimlen()	pqueue_inq_dimlen()
ncmpi_inq_var()	pqueue_inq_var()
ncmpi_inq_varid()	pqueue_inq_varid()
ncmpi_inq_varname()	pqueue_inq_varname()
ncmpi_inq_varndims()	pqueue_inq_varndims()
ncmpi_inq_nvars()	pqueue_inq_nvars()

We discuss the implementation of the PQueue API and the optimizations performed to reduce memory read/write latency and improve efficiency of our split PQueue design in the next section.

4.2 PQueue Implementation

Figure 3 depicts the amount of time taken to create (yellow), open (cyan), and close (magenta) a persistent memory pool using PMDK and the amount of time taken for allocating (green), reading from (blue), and writing (red) to persistent memory. The X-axis in the figure corresponds to the data size in Megabytes (MB), and the Y-axis corresponds to the time in seconds. Note that while the cost associated with creating a pool is not dependent upon the pool size. Moreover, when dealing with large data, the cost associated with reading data is significantly lower than that associated with writing the data [13]. Based on the above observations, we implement the following two optimizations.

4.2.1 DRAM Buffering. The function `pqueue_var_write*` is used to write data, and the function `pqueue_flush_entry` is responsible for syncing the written data with the persistent memory. The data passed to the function `pqueue_var_write*` is stored in an internal DRAM buffer rather than initiating the opening and closing of the pool for every write call until the writer process invokes `pqueue_flush_entry` function. The function `pqueue_flush_entry`

Table 2: PnetCDF and PQueue Attribute and Data Mode APIs

Attribute APIs	
PnetCDF APIs	PQueue APIs
ncmpi_inq_att()	pqueue_inq_att()
ncmpi_inq_attid()	pqueue_inq_attid()
ncmpi_inq_atttype()	pqueue_inq_atttype()
ncmpi_inq_attlen()	pqueue_inq_attlen()
ncmpi_inq_attname()	pqueue_inq_attname()
ncmpi_put_att_text()	pqueue_put_att_text()
ncmpi_get_att_text()	pqueue_get_att_text()
ncmpi_put_att_int()	pqueue_put_att_int()
ncmpi_get_att_int()	pqueue_get_att_int()
ncmpi_put_att_float()	pqueue_put_att_float()
ncmpi_get_att_float()	pqueue_get_att_float()
ncmpi_put_att_double()	pqueue_put_att_double()
ncmpi_get_att_double()	pqueue_get_att_double()
Data Mode APIs	
ncmpi_put_var_text()	pqueue_var_write_text()
ncmpi_put_var_int()	pqueue_var_write_int()
ncmpi_put_var_float()	pqueue_var_write_float()
ncmpi_put_var_double()	pqueue_var_write_double()
ncmpi_get_var_text()	pqueue_var_read_text()
ncmpi_get_var_int()	pqueue_var_read_int()
ncmpi_get_var_float()	pqueue_var_read_float()
ncmpi_get_var_double()	pqueue_var_read_double()
ncmpi_put_vara_text()	pqueue_vara_write_text()
ncmpi_put_vara_int()	pqueue_vara_write_int()
ncmpi_put_vara_float()	pqueue_vara_write_float()
ncmpi_put_vara_double()	pqueue_vara_write_double()
ncmpi_get_vara_text()	pqueue_vara_read_text()
ncmpi_get_vara_int()	pqueue_vara_read_int()
ncmpi_get_vara_float()	pqueue_vara_read_float()
ncmpi_get_vara_double()	pqueue_vara_read_double()

flushes the buffer data to persistent memory, hence making the data persistent for future access. Similarly, in the context of the reading process, all variables are obtained during the initial request for reading and subsequently stored in a buffer of dynamic random-access memory (DRAM). The buffer acts as storage for handling subsequent read requests by accessing the appropriate variables. The function `pqueue_next_entry` clears the buffer upon its invocation.

4.2.2 Domain Decomposition. The number of processes used to execute the simulation and analysis applications are often different and may vary as per the problem size. The analysis tasks typically require fewer processes, and the number of simulation processes is significantly higher because the simulations are more compute-intensive. The domain may be decomposed in various ways, one of the ways to decompose a domain by the writers (simulation processes) is depicted in Figure 4. $P_0, P_1, \dots$ represent the various simulation processes. The decomposition for analysis (readers) may differ from the domain decomposition of the simulation processes. The simulation processes (writers) directly write their respective subdomains to the corresponding entries during the writing phase. The readers compute the offsets depending on the decomposition

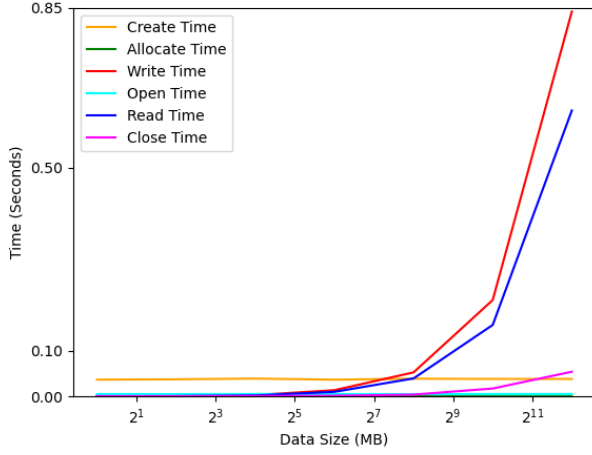


Figure 3: Time (seconds) comparison of create, open, close, allocate, write and read operations provided by libpmemobj

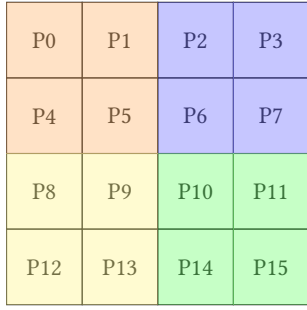


Figure 4: Domain decomposition in simulation and analysis applications. In this example, the 2D domain is decomposed into 16 (4×4) subdomains by the simulation application (processes P0 to P16) and 4 subdomains by the analysis application (every subdomain is shown in a different color).

and accordingly fetch data from PQueue entries. The readers utilize additional metadata (process identifier) added to the entries to fetch data from the correct sub-domains. For example, the reader process 0 in Figure 4 fetches data from entries of simulation processes P0, P1, P4, and P5. The mapping between simulation processes (writers) and analysis processes (readers) is shown in Figure 5. The four readers (A0, A1, A2, A3) are shown in different colours for the 4×4 simulation process grid.

We summarize the end-to-end process of writing data and reading data using PQueue methods as follows:

- (1) The `pqueue_var_write*` function writes the variable data to a local DRAM buffer and stores it as a cache. The data in the DRAM buffer is only persistent once flushed to persistent memory PMEM.
- (2) `pqueue_flush_entry` function writes the data cached in the DRAM buffer to persistent memory and empties the DRAM buffer. `pqueue_flush_entry` is called when the writer has

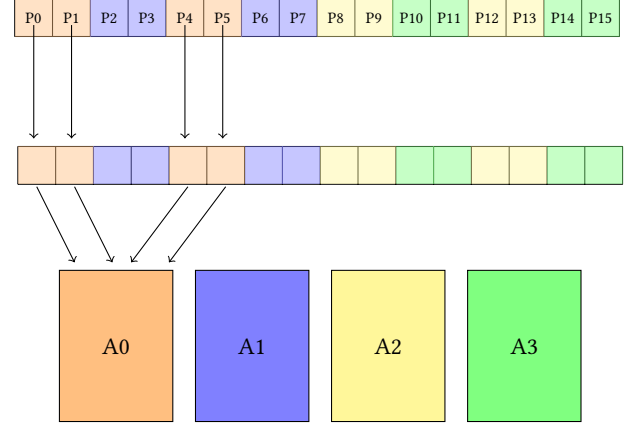


Figure 5: Sub-domain association while reading data. Readers use the metadata information to correctly fetch data corresponding to their sub-domains. The arrows depict the data flow from the corresponding entries to analysis process A0.

finished writing all the variable data. This also sets the flag that informs the readers that they can start reading the data.

- (3) While reading data, `pqueue_var_read*` first copies all the variable data from persistent memory to a local DRAM buffer.
- (4) Once the data is available in the local buffer, it rearranges it as per the domain decomposition and performs analysis.
- (5) Once the reader finishes reading a time step entry, the reader calls the function `pqueue_next_entry` to access the subsequent time step data, and the function clears the reader DRAM buffer and marks the queue entry as empty before proceeding to the next time step.

4.3 PQueue Runtime

The PQueue library utilizes the libpmemobj library [2] for performing persistent memory operations. Persistent Memory Development Kit (PMDK) needs to be installed on the nodes. The PQueue library can be loaded by specifying the library's path using the LD_PRELOAD mechanism. This will supersede the existing PnetCDF functions and will eliminate the need for application developers to make any modifications to the source code. Whenever a PnetCDF function is called, it will internally execute the corresponding PQueue function. It is important to highlight that both the PnetCDF and PQueue libraries can be utilized independently of each other. It should be noted that PnetCDF functions are not inherently associated with PQueue. Users can explicitly link PQueue as well as PnetCDF in order to use both libraries for different purposes. Users need to specify the persistent memory path by utilizing the macro `PMEM_FNAME`. The data generated by the application will be stored in the configured path.

5 EXPERIMENTS AND RESULTS

In this section, we present the performance evaluation of PQueue as compared to PnetCDF. For the experiments, we used an Intel Xeon Gold 6252N node operating at a frequency of 2.30 GHz CPU.

The processor comprises 96 cores evenly distributed between two non-uniform memory access (NUMA) nodes. We used a 256 GB capacity Intel Optane persistent memory device operating in device direct access mode (DAX) and a 2 TB Toshiba 6 Gbit/s SATA hard disk drive. We evaluate using a micro-benchmark comprising reader and writer processes which perform input/output operations. The writer also performs minor computations to mimic simulations.

5.1 Micro-benchmark

```
int main()
{
    double start_time = 0;
    double end_time = 0;
    start_time = MPI_Wtime(); // Start timer
    for(int iterations = 0; iterations < 20; iterations++)
    {
        // Perform read operations.
    }
    end_time = MPI_Wtime() - start_time; // Read time
    return 0;
}
```

Listing 1: Reader Benchmark Example

```
int main()
{
    double start_time = 0;
    double end_time = 0;
    double write_time = 0;
    // initial setup, define variables, dimensions
    for(int iterations = 0; iterations < 20; iterations++)
    {
        perform small computations.
        start_time = MPI_Wtime();
        // Perform write operations.
        end_time = MPI_Wtime();
        // accumulate write time
        write_time += (end_time - start_time);
    }
    return 0;
}
```

Listing 2: Writer Benchmark Example

The workload comprises a reader and a writer application as shown in Listings 1 and 2. This mimics simulations (writers) and analysis (readers). Each application may utilize multiple processes. There may be multiple writers and readers and the ratio of writers to readers may vary too. The domain decompositions for both are specified by the user. Both applications commence simultaneously. The variables are assigned random values during the initialization process. The writers perform arithmetic operations on the grid variables in each iteration and store the outcome in the designated storage (PnetCDF or PQueue). The readers systematically process each variable and then proceed to the next iteration.

In case of PQueue, the data is processed concurrently by the reader. The reader can start reading as soon as one time-step entry is complete. In case of PnetCDF, the reader initiates the reading process after all writers complete writing the output for a time step. This is similar to the design of analysis applications that read NetCDF files. However, note that in most post processing applications, the readers read after all output time steps are written to the file. We modified the workflow to do a fair comparison between PQueue and PnetCDF. The readers read as soon as one time step is written to disk/memory as the case may be.

5.2 Evaluation Metrics

We experimented with different data sizes (ranging from 2^6 to 2^{12} megabytes), six unique reader-writer combinations, and 20 output steps. The writer application is executed on 4, 8, and 16 MPI processes, whereas the reader application is executed on 2 and 4 MPI processes. We show error bars as the standard deviation of the execution times obtained from three separate runs. The PQueue data structure had a predefined queue size of 10 entries.

We link our micro-benchmark reader and writer applications either with PnetCDF or with PQueue and measure the respective performance. We used two timers for timing the read times and write times as shown in Listings 1 and 2:

- (1) Write simulation timer: The write simulation timer considers only the time required to transfer the data to PQueue or PnetCDF. The timer starts before writing the data to the disk or persistent memory and stops once the write is completed. The process repeats for 20 iterations and calculates the total time.
- (2) End-to-end read timer: The end-to-end read timer starts before opening the corresponding file or memory pool and stops when the data reads 20 iterations.

5.3 Results

Figures 6, 7, 8, and 9 show the timing comparisons between PQueue and PnetCDF for six distinct combinations of readers (2, 4) and writers (4, 8, 16) and varying data sizes (up to 4 GB). Generally simulations are executed using higher number of processes and analysis is performed using fewer processes. Thus we chose these configurations. The X-axis represents data sizes in megabytes (MB), while the Y-axis represents time in seconds. The timings were generated by the two timers – end-to-end read timer and a simulation write timer, and are plotted for different data sizes using Pnetcdf and PQueue in every figure. The errors bars show the variation from three runs in every plot.

5.3.1 Write time analysis. Figures 6, 7 show the write times for PQueue (blue) and PnetCDF (red). Figure 6 show the times when there are 2 readers and various number of writers (4, 8, 16). Figure 7 show the times when there are 4 readers and various number of writers (4, 8, 16). The X-axis shows the various data sizes (64 MB – 4 GB). In both cases we observe that the write times increase for PQueue (blue) and PnetCDF (red) with increase in data size. We also observe decrease in the time required to write data with increase in number of writer processes. This is observed in both the PQueue and PnetCDF. However, there is a significant difference in time for larger data sizes between PQueue and PnetCDF. We observe that

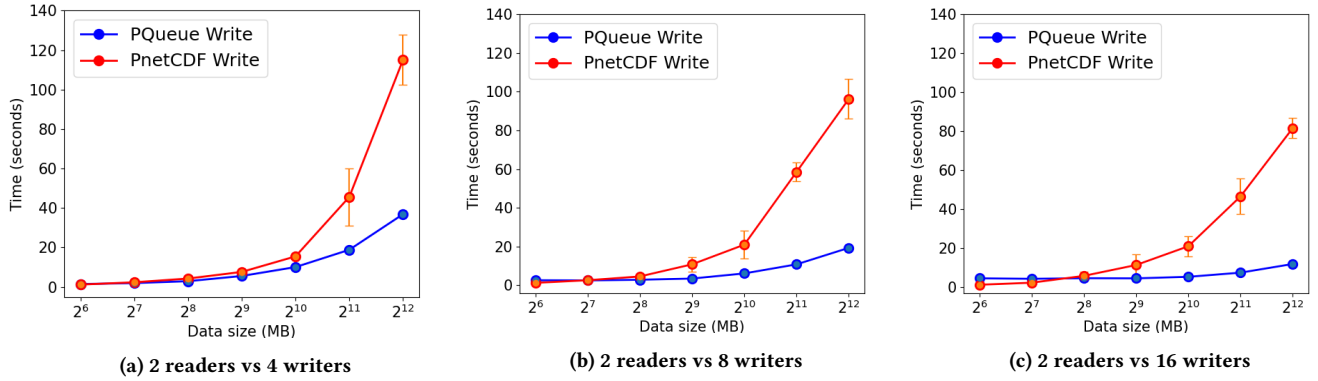


Figure 6: Comparison of write time (seconds) for 2 readers vs. 4 writers, 8 writers, and 16 writers respectively. The X-axis represents data sizes in megabytes (MB), while the Y-axis represents time in seconds. The timing data generated by simulation write timer is plotted for different data sizes using Pnetcdf and PQueue respectively. The error bars are generated by calculating the standard deviation of the timers obtained from three separate runs.

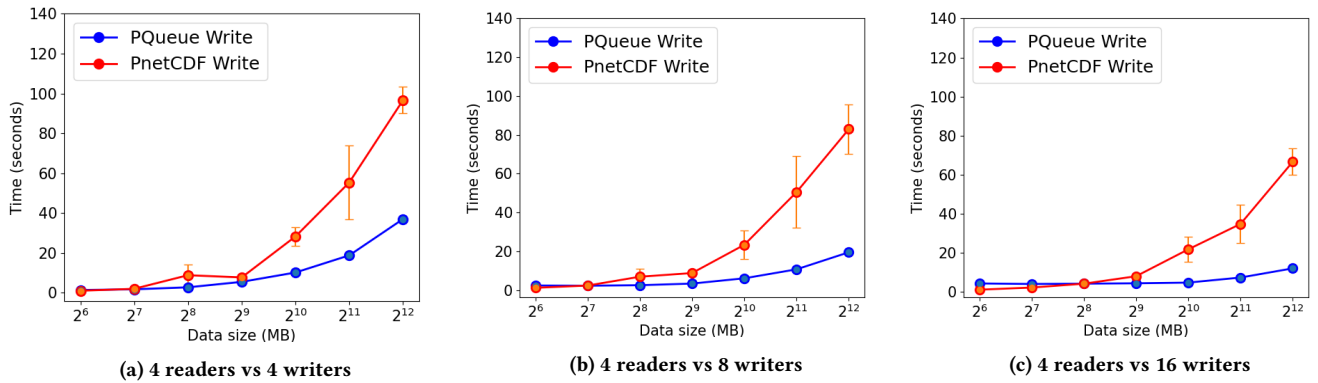


Figure 7: Comparison of write time (seconds) for 4 readers vs. 4 writers, 8 writers, and 16 writers respectively. The X-axis represents data sizes in megabytes (MB), while the Y-axis represents time in seconds. The timing data generated by simulation write timer is plotted for different data sizes using Pnetcdf and PQueue respectively. The error bars are generated by calculating the standard deviation of the timers obtained from three separate runs.

the gap between PQueue and PnetCDF performance increases with data size, i.e. the effect of I/O bottleneck is more pronounced in case of PnetCDF which writes to a NetCDF file in parallel. Note that the performance with serial NetCDF may have been worse and hence not shown here.

When considering two readers, it is evident from Figure 6 that there are notable reductions in write times for 4, 8, and 16 writers, by up to a factor of 3 $\times$, 5.5 $\times$, and 7 $\times$ respectively. The reduction in write times with PQueue in case of 4, 8 and 16 writers were by a factor of 3.2 $\times$, 4.7 $\times$, and 5.5 $\times$ respectively. This shows the utility of our library PQueue for in situ analysis of simulation output where the prohibitive high I/O cost may be mitigated using non-volatile memory.

5.3.2 Read time analysis. Figures 8, 9 show the read times with PQueue (blue) and PnetCDF (red). We run the reader application in

parallel with the writer. In case of PQueue, the readers read from the NVM whereas in case PnetCDF, the readers read from the hard disk. Figure 8 show the times when there are 2 readers and various number of writers (4, 8, 16). Figure 9 show the times when there are 4 readers and various number of writers (4, 8, 16). The X-axis shows the various data sizes (64 MB – 4 GB).

We observed a decrease in read times by 2 $\times$, 4 $\times$ and 6 $\times$ for 4, 8 and 16 writers respectively in case of PQueue as compared to PnetCDF with 2 readers. We observed a decrease in read times by 3 $\times$, 5 $\times$ and 10 $\times$ for 4, 8 and 16 writers respectively in case of PQueue as compared to PnetCDF with 4 readers. The figures also demonstrate a clear trend between Pnetcdf time and PQueue times; both increase with increase in data sizes. The increase is significantly higher in case of 4 readers with PnetCDF. This is because the readers need to wait for the writers to complete writing a time step, which is more in case of PnetCDF.

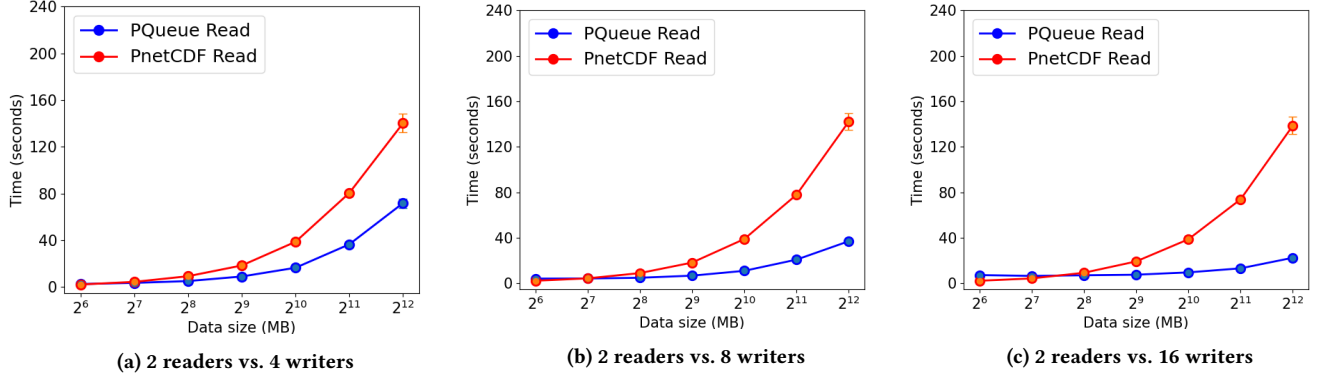


Figure 8: Comparison of read time (seconds) for 2 readers vs. 4 writers, 8 writers, and 16 writers respectively. The X-axis represents data sizes in megabytes (MB), while the Y-axis represents time in seconds. The timing data generated by end-to-end read timer is plotted for different data sizes using Pnetcdf and PQueue respectively. The error bars are generated by calculating the standard deviation of the timers obtained from three separate runs.

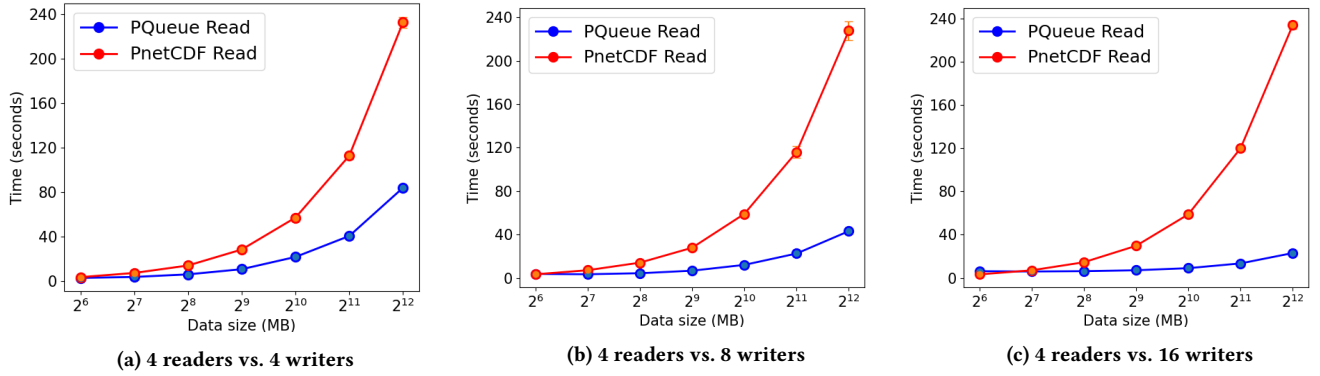


Figure 9: Comparison of read time (seconds) for 4 readers vs. 4 writers, 8 writers, and 16 writers respectively. The X-axis represents data sizes in megabytes (MB), while the Y-axis represents time in seconds. The timing data generated by end-to-end read timer is plotted for different data sizes using Pnetcdf and PQueue respectively. The error bars are generated by calculating the standard deviation of the timers obtained from three separate runs.

We observe that the read times remain almost constant with increase in the number of writers (4, 8, 16) in case of PnetCDF. This is because the writers write to a shared NetCDF file and it does not impact the read times. In contrast, the read time decreases for the same data size with increase in the number of writers in case of PQueue. This is because of our split-PQueue design where multiple writers can simultaneously update entries. Increase in number of writers reduce the write times and thereby the readers can quickly access the data as soon as the time step is written by the writers, reducing the wait times in the readers. With increase in the number of reader processes, we observe that the time required to read data increases in case of PnetCDF. This may be due to contention. Increase in the number of readers did not impact the read time much in case of PQueue.

6 CONCLUSIONS

Scientific simulations generate massive amount of output which requires to be analyzed. Writing output to the disk and reading back from the disk for analysis has been prohibitively costly now due to slower disk speeds as compared to compute rates. Most existing I/O data transfer libraries, such as PnetCDF, used in scientific simulations, use POSIX calls internally to write to disk. However, recent high-speed memory technologies such as Intel Optane memory present an opportunity to leverage these advancements for data transfer between simulations and analysis. We presented PQueue, an end-to-end, high-speed data transfer library designed for in situ analysis of scientific simulations. We also presented split-PQueue that resolves the limitations posed by PMDK in order to enable simultaneous reads and writes by several processes to the persistent memory. PQueue provides an Application Programming Interface

(API) that resembles high-level parallel Input/Output (I/O) libraries such as Parallel NetCDF (PnetCDF). This enables application developers seamlessly use PQueue by simply dynamic linking the same application that uses NetCDF/PnetCDF APIs to perform I/O. We performed a comparative analysis to evaluate the concurrent data reading and writing capabilities of PQueue with PnetCDF. We observed up to 7× decrease in data write times and 10× decrease in read times with 4 GB data size using up to 16 writers and 4 readers.

In future, we plan to experiment with real scientific applications. We also plan to test and scale on multiple nodes. The current design will seamlessly work for scientific applications that output NetCDF files. PQueue is a node-local structure, hence it is straight-forward to run on multiple nodes where readers are on the same node as their respective writers. Although our design is scalable, extensive testing on multiple nodes remains to be done. In future, we will also consider scenarios where readers and writers are placed on different nodes and hence do not share the node-local storage (PMEM).

ACKNOWLEDGMENTS

This work is supported by the Science and Engineering research board (SERB), Department of Science and Technology (DST), Government of India under the SERB Start-up Research Grant (SRG/2020/002503) to Preeti Malakar. We thank Paras Mittal for his initial work on PQueue. We also thank the Department of Computer Science and Engineering, IIT Kanpur for providing access to computational facilities.

REFERENCES

- [1] 2017. *Introduction to Persistent Memory Programming*. Intel Corporation. <https://www.intel.com/content/www/us/en/developer/topic-technology/persistent-memory/overview.html>
- [2] 2023. *libpmemobj - persistent memory transactional object store*. <https://pmem.io/pmdk/manpages/windows/master/libpmemobj/libpmemobj.7/>
- [3] 2023. *Persistent Memory Development Kit (PMDK)*. <https://pmem.io/pmdk/>
- [4] 2023. *pmem.io github*. <https://github.com/pmem/pmdk/issues/4141>. <https://github.com/pmem/pmdk/issues/4141>
- [5] 2023. *Top 500 Supercomputers*. <https://top500.org>
- [6] Sadaf R. Alam, Hussein N. El-Harake, Kristopher Howard, Neil Stringfellow, and Fabio Verzelloni. 2011. Parallel I/O and the Metadata Wall. In *Proceedings of the Sixth Workshop on Parallel Data Storage* (Seattle, Washington, USA) (PDSW '11). Association for Computing Machinery, New York, NY, USA, 13–18.
- [7] Utkarsh Ayachit, Andrew Bauer, Earl P. N. Duque, Greg Eisenhauer, Nicola Ferrier, Junmin Gu, Kenneth E. Jansen, Burt Loring, Zarija Lukic, Suresh Menon, Dmitriy Morozov, Patrick O'Leary, Reetesh Ranjan, Michel Rasquin, Christopher P. Stone, Venkat Vishwanath, Gunther H. Weber, Brad Whitlock, Matthew Wolf, K. John Wu, and E. Wes Bethel. 2016. Performance Analysis, Design Considerations, and Applications of Extreme-Scale In Situ Infrastructures. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 921–932. <https://doi.org/10.1109/SC.2016.78>
- [8] Julian Borrill, Leonid Oliker, John Shalf, and Hongzhang Shan. 2007. Investigation of Leading HPC I/O Performance Using a Scientific-Application Derived Benchmark. In *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing* (Reno, Nevada) (SC '07). Association for Computing Machinery, New York, NY, USA, Article 10, 12 pages. <https://doi.org/10.1145/1362622.1362636>
- [9] Hank Childs, Sean D. Ahern, James Ahrens, Andrew C. Bauer, Janine Bennett, E. Wes Bethel, Peer-Timo Bremer, Eric Brugger, Joseph Cottam, Matthieu Dorier, Soumya Dutta, Jean M. Favre, Thomas Fogal, Steffen Frey, Christoph Garth, Berk Geveci, William F. Godoy, Charles D. Hansen, Cyrus Harrison, Bernd Hentschel, Joseph Insley, Chris R. Johnson, Scott Klasky, Aaron Knoll, James Kress, Matthew Larsen, Jay Lofstead, Kwan-Liu Ma, Preeti Malakar, Jeremy Meredith, Kenneth Moreland, Paul Navrátil, Patrick O'Leary, Manish Parashar, Valerio Pascucci, John Patchett, Tom Peterka, Steve Petruzza, Norbert Podhorski, David Pugmire, Michel Rasquin, Silvio Rizzi, David H. Rogers, Sudhanshu Sane, Franz Sauer, Robert Sisneros, Han-Wei Shen, Will Usher, Rhonda Vickery, Venkatram Vishwanath, Ingo Wald, Ruonan Wang, Gunther H. Weber, Brad Whitlock, Matthew Wolf, Hongfeng Yu, and Sean B. Ziegler. 2020. A terminology for in situ visualization and analysis systems. *The International Journal of High Performance Computing Applications* 34, 6 (2020), 676–691.
- [10] Shaohua Duan, Pradeep Subedi, Philip Davis, Keita Teranishi, Hemanth Kolla, Marc Gamell, and Manish Parashar. 2020. CoREC: Scalable and resilient in-memory data staging for in-situ workflows. *ACM Transactions on Parallel Computing (TOPC)* 7, 2 (2020), 1–29.
- [11] Mike Folk, Gerd Heber, Quincey Koziol, Elena Pourmal, and Dana Robinson. 2011. An overview of the HDF5 technology suite and its applications. *ACM International Conference Proceeding Series*, 36–47.
- [12] Swapnil Haria, Mark D Hill, and Michael M Swift. 2020. MOD: Minimally ordered durable datastructures for persistent memory. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 775–788.
- [13] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amir Saman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, and Steven Swanson. 2019. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module. *CoRR* abs/1903.05714 (2019). arXiv:1903.05714 <http://arxiv.org/abs/1903.05714>
- [14] Maciej Jakubowski and Piotr Sypek. 2022. Electromagnetic Simulations with 3D FEM and Intel Optane Persistent Memory. In *2022 24th International Microwave and Radar Conference (MIKON)*. 1–5.
- [15] Tong Jin, Fan Zhang, Qian Sun, Hoang Bui, Melissa Romanus, Norbert Podhorski, Scott Klasky, Hemanth Kolla, Jacqueline Chen, Robert Hager, et al. 2015. Exploring data staging across deep memory hierarchies for coupled data intensive simulation workflows. In *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 1033–1042.
- [16] Awais Khan, Hyogi Sim, Sudharshan S. Vazhkudai, and Youngjae Kim. 2021. MOSIQS: Persistent Memory Object Storage With Metadata Indexing and Querying for Scientific Computing. *IEEE Access* 9 (2021), 85217–85231. <https://doi.org/10.1109/ACCESS.2021.3087502>
- [17] Jianwei Li, Wei-keng Liao, Alok Choudhary, Robert Ross, Rajeev Thakur, William Gropp, Rob Latham, Andrew Siegel, Brad Gallagher, and Michael Zingale. 2003. Parallel NetCDF: A High-Performance Scientific I/O Interface. In *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing* (Phoenix, AZ, USA) (SC '03). Association for Computing Machinery, 39.
- [18] Onur Mutlu and L. Subramanian. 2014. Research Problems and Opportunities in Memory Systems. *Supercomputing Frontiers and Innovations* 1 (09 2014), 19–55. <https://doi.org/10.14529/jsfi140302>
- [19] Onkar Patil, Latchesar Ionkov, Jason Lee, Frank Mueller, and Michael Lang. 2019. Performance Characterization of a DRAM-NVM Hybrid Memory Architecture for HPC Applications Using Intel Optane DC Persistent Memory Modules. In *Proceedings of the International Symposium on Memory Systems* (Washington, District of Columbia, USA) (MEMSYS '19). Association for Computing Machinery, New York, NY, USA, 288–303. <https://doi.org/10.1145/3357526.3357541>
- [20] Jordan Powers, Joseph Klemp, William Skamarock, Christopher Davis, Jimmy Duda, David Gill, Janice Coen, David Gochis, Ravan Ahmadov, Steven Peckham, Georg Grell, John Michalak, Samuel Trahan, Stanley Benjamin, Curtis Alexander, Geoffrey Dimego, Wei Wang, Craig Schwartz, Glen Romine, and Michael Duda. 2017. The Weather Research and Forecasting (WRF) Model: Overview, System Efforts, and Future Directions. *Bulletin of the American Meteorological Society* 98 (01 2017). <https://doi.org/10.1175/BAMS-D-15-00308.1>
- [21] Jie Ren, Jiaolin Luo, Ivy Peng, Kai Wu, and Dong Li. 2021. Optimizing Large-Scale Plasma Simulations on Persistent Memory-Based Heterogeneous Memory with Effective Data Placement across Memory Hierarchy. In *Proceedings of the ACM International Conference on Supercomputing* (ICS '21). Association for Computing Machinery, New York, NY, USA.
- [22] R. Rew and G. Davis. 1990. NetCDF: an interface for scientific data access. *IEEE Computer Graphics and Applications* 10, 4 (1990), 76–82. <https://doi.org/10.1109/38.56302>
- [23] Steve Scargall. 2020. *Introduction to Persistent Memory Programming*. https://doi.org/10.1007/978-1-4842-4932-1_1
- [24] Pradeep Subedi, Philip Davis, Shaohua Duan, Scott Klasky, Hemanth Kolla, and Manish Parashar. 2018. Stacker: An Autonomic Data Movement Engine for Extreme-Scale Data Staging-Based In-Situ Workflows. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. 920–930. <https://doi.org/10.1109/SC.2018.00076>
- [25] Ananta Tiwari, Anthony Gamst, Michael A. Laurenzano, Martin Schulz, and Laura Carrington. 2014. Modeling the Impact of Reduced Memory Bandwidth on HPC Applications. In *Euro-Par 2014 Parallel Processing*, Fernando Silva, Inês Dutra, and Vitor Santos Costa (Eds.). Springer International Publishing, Cham, 63–74.
- [26] Kan Wu, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2019. Towards an Unwritten Contract of Intel Optane SSD. In *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*. USENIX Association, Renton, WA. <https://www.usenix.org/conference/hotstorage19/presentation/wu-kan>